

conditional operators and third way is to use the **switch** statement.

- (b) The default scope of the **if** statement is only the next statement. So, to execute more than one statement they must be written in a pair of braces.
- (c) An **if** block need not always be associated with an **else** block. However, an **else** block is always associated with an **if** statement.
- (d) If the outcome of an **if-else** ladder is only one of two answers then the ladder should be replaced either with an **else-if** clause or by logical operators.
- (e) **&&** and **||** are binary operators, whereas, **!** is a unary operator.
- (f) In C every test expression is evaluated in terms of zero and non-zero values. A zero value is considered to be false and a non-zero value is considered to be true.
- (g) Assignment statements used with conditional operators must be enclosed within a pair of parenthesis.

Exercise

if, if-else, Nested if-elses

[A] What would be the output of the following programs:

```
(a)  main()  
    {  
        int  a = 300, b, c ;  
        if ( a >= 400 )  
            b = 300 ;  
            c = 200 ;  
        printf ( "\n%d %d", b, c ) ;  
    }
```

```
(b)  main()  
    {  
        int  a = 500, b, c ;  
        if ( a >= 400 )
```

```
        b = 300 ;
        c = 200 ;
        printf ( "\n%d %d", b, c ) ;
    }

(c)  main()
    {
        int  x = 10, y = 20 ;
        if ( x == y ) ;
            printf ( "\n%d %d", x, y ) ;
    }

(d)  main()
    {
        int  x = 3, y = 5 ;
        if ( x == 3 )
            printf ( "\n%d", x ) ;
        else ;
            printf ( "\n%d", y ) ;
    }

(e)  main()
    {
        int  x = 3 ;
        float y = 3.0 ;

        if ( x == y )
            printf ( "\nx and y are equal" ) ;
        else
            printf ( "\nx and y are not equal" ) ;
    }

(f)  main()
    {
        int  x = 3, y, z ;
        y = x + 10 ;
        z = x < 10 ;
        printf ( "\nx = %d y = %d z = %d", x, y, z ) ;
    }
```

```
(g)  main()  
    {  
        int k = 35 ;  
        printf ( "\n%d %d %d", k == 35, k = 50, k > 40 ) ;  
    }
```

```
(h)  main()  
    {  
        int i = 65 ;  
        char j = 'A' ;  
        if ( i == j )  
            printf ( "C is WOW" ) ;  
        else  
            printf( "C is a headache" ) ;  
    }
```

```
(i)  main()  
    {  
        int  a = 5, b, c ;  
        b = a = 15 ;  
        c = a < 15 ;  
        printf ( "\na = %d b = %d c = %d", a, b, c ) ;  
    }
```

```
(j)  main()  
    {  
        int x = 15 ;  
        printf ( "\n%d %d %d", x != 15, x = 20, x < 30 ) ;  
    }
```

[B] Point out the errors, if any, in the following programs:

```
(a)  main()  
    {  
        float  a = 12.25, b = 12.52 ;  
        if ( a = b )  
            printf ( "\na and b are equal" ) ;  
    }
```

- ```
 }
```
- (b) `main()`
- ```
{
    int j = 10, k = 12 ;
    if ( k >= j )
    {
        {
            k = j ;
            j = k ;
        }
    }
}
```
- (c) `main()`
- ```
{
 if ('X' < 'x')
 printf ("\nascii value of X is smaller than that of x");
}
```
- (d) `main()`
- ```
{
    int x = 10 ;
    if ( x >= 2 ) then
        printf ( "\n%d", x );
}
```
- (e) `main()`
- ```
{
 int x = 10 ;
 if x >= 2
 printf ("\n%d", x);
}
```
- (f) `main()`
- ```
{
    int x = 10, y = 15 ;
    if ( x % 2 = y % 3 )
```

```
        printf ( "\nCarpathians" ) ;
    }

(g)  main()
    {
        int x = 30 , y = 40 ;
        if ( x == y )
            printf( "x is equal to y" ) ;
        elseif ( x > y )
            printf( "x is greater than y" ) ;
        elseif ( x < y )
            printf( "x is less than y" ) ;
    }

(h)  main()
    {
        int x = 10 ;
        if ( x >= 2 ) then
            printf ( "\n%d", x ) ;
    }

(i)  main()
    {
        int a, b ;
        scanf ( "%d %d",a, b ) ;
        if ( a > b ) ;
            printf ( "This is a game" ) ;
        else
            printf ( "You have to play it" ) ;
    }
```

[C] Attempt the following:

- (a) If cost price and selling price of an item is input through the keyboard, write a program to determine whether the seller has made profit or incurred loss. Also determine how much profit he made or loss he incurred.

- (b) Any integer is input through the keyboard. Write a program to find out whether it is an odd number or even number.
- (c) Any year is input through the keyboard. Write a program to determine whether the year is a leap year or not.
(Hint: Use the % (modulus) operator)
- (d) According to the Gregorian calendar, it was Monday on the date 01/01/1900. If any year is input through the keyboard write a program to find out what is the day on 1st January of this year.
- (e) A five-digit number is entered through the keyboard. Write a program to obtain the reversed number and to determine whether the original and reversed numbers are equal or not.
- (f) If the ages of Ram, Shyam and Ajay are input through the keyboard, write a program to determine the youngest of the three.
- (g) Write a program to check whether a triangle is valid or not, when the three angles of the triangle are entered through the keyboard. A triangle is valid if the sum of all the three angles is equal to 180 degrees.
- (h) Find the absolute value of a number entered through the keyboard.
- (i) Given the length and breadth of a rectangle, write a program to find whether the area of the rectangle is greater than its perimeter. For example, the area of the rectangle with length = 5 and breadth = 4 is greater than its perimeter.
- (j) Given three points **(x1, y1)**, **(x2, y2)** and **(x3, y3)**, write a program to check if all the three points fall on one straight line.

- (k) Given the coordinates (**x, y**) of a center of a circle and it's radius, write a program which will determine whether a point lies inside the circle, on the circle or outside the circle.

(Hint: Use **sqrt()** and **pow()** functions)

- (l) Given a point (**x, y**), write a program to find out if it lies on the x-axis, y-axis or at the origin, viz. (0, 0).

Logical Operators

If $a = 10$, $b = 12$, $c = 0$, find the values of the expressions in the following table:

Expression	Value
$a != 6 \ \&\& \ b > 5$ $a == 9 \ \ b < 3$ $! (a < 10)$ $! (a > 5 \ \&\& \ c)$ $5 \ \&\& \ c != 8 \ \ !c$	1

[D] What would be the output of the following programs:

- (a)

```
main( )
{
    int i = 4, z = 12 ;
    if ( i = 5 || z > 50 )
        printf ( "\nDean of students affairs" ) ;
    else
        printf ( "\nDosa" ) ;
}
```

- (b)

```
main( )
{
    int i = 4, z = 12 ;
```

```
        if ( i = 5 && z > 5 )
            printf ( "\nLet us C" ) ;
        else
            printf ( "\nWish C was free !" ) ;
    }

(c)  main( )
    {
        int  i = 4, j = -1, k = 0, w, x, y, z ;
        w = i || j || k ;
        x = i && j && k ;
        y = i || j && k ;
        z = i && j || k ;
        printf ( "\nw = %d x = %d y = %d z = %d", w, x, y, z ) ;
    }

(d)  main( )
    {
        int  i = 4, j = -1, k = 0, y, z ;
        y = i + 5 && j + 1 || k + 2 ;
        z = i + 5 || j + 1 && k + 2 ;
        printf ( "\ny = %d z = %d", y, z ) ;
    }

(e)  main( )
    {
        int  i = -3, j = 3 ;
        if ( !i + !j * 1 )
            printf ( "\nMassaro" ) ;
        else
            printf ( "\nBennarivo" ) ;
    }

(f)  main( )
    {
        int  a = 40 ;
        if ( a > 40 && a < 45 )
            printf ( "a is greater than 40 and less than 45" ) ;
    }
```



```
        else
            printf ( "%d", a ) ;
    }
```

```
(g)  main()
    {
        int  p = 8, q = 20 ;
        if ( p == 5 && q > 5 )
            printf ( "\nWhy not C" ) ;
        else
            printf ( "\nDefinitely C !" ) ;
    }
```

```
(h)  main()
    {
        int i = -1, j = 1, k ,l ;
        k = i && j ;
        l = i || j ;
        printf ( "%d %d", l, j ) ;
    }
```

```
(i)  main()
    {
        int x = 20 , y = 40 , z = 45 ;
        if ( x > y && x > z )
            printf( "x is big" ) ;
        else if ( y > x && y > z )
            printf( "y is big" ) ;
        else if ( z > x && z > y )
            printf( "z is big" ) ;
    }
```

```
(j)  main()
    {
        int i = -1, j = 1, k ,l ;
        k = !i && j ;
        l = !i || j ;
        printf ( "%d %d", i, j ) ;
    }
```

```
}
```

```
(k)  main()  
    {  
        int  j = 4, k ;  
        k = !5 && j ;  
        printf ( "\nk = %d", k ) ;  
    }
```

[E] Point out the errors, if any, in the following programs:

```
(a)  /* This program  
    /* is an example of  
    /* using Logical operators */  
    main()  
    {  
        int  i = 2, j = 5 ;  
        if ( i == 2 && j == 5 )  
            printf ( "\nSatisfied at last" ) ;  
    }
```

```
(b)  main()  
    {  
        int  code, flag ;  
        if ( code == 1 & flag == 0 )  
            printf ( "\nThe eagle has landed" ) ;  
    }
```

```
(c)  main()  
    {  
        char  spy = 'a', password = 'z' ;  
        if ( spy == 'a' or password == 'z' )  
            printf ( "\nAll the birds are safe in the nest" ) ;  
    }
```

```
(d)  main()  
    {
```

```
int i = 10, j = 20 ;
if ( i = 5 ) && if ( j = 10 )
    printf ( "\nHave a nice day" ) ;
}
```

```
(a) main()
{
    int x = 10 , y = 20;
    if ( x >= 2 and y <=50 )
        printf ( "\n%d", x ) ;
}
```

```
(b) main()
{
    int a, b ;
    if ( a == 1 & b == 0 )
        printf ( "\nGod is Great" ) ;
}
```

```
(c) main()
{
    int x = 2;
    if ( x == 2 && x != 0 ) ;
    {
        printf ( "\nHi" ) ;
        printf( "\nHello" ) ;
    }
    else
        printf( "Bye" ) ;
}
```

```
(d) main()
{
    int i = 10, j = 10 ;
    if ( i && j == 10)
        printf ( "\nHave a nice day" ) ;
}
```

[F] Attempt the following:

- (a) Any year is entered through the keyboard, write a program to determine whether the year is leap or not. Use the logical operators **&&** and **||**.
- (b) Any character is entered through the keyboard, write a program to determine whether the character entered is a capital letter, a small case letter, a digit or a special symbol.

The following table shows the range of ASCII values for various characters.

Characters	ASCII Values
A – Z	65 – 90
a – z	97 – 122
0 – 9	48 – 57
special symbols	0 - 47, 58 - 64, 91 - 96, 123 - 127

- (c) An Insurance company follows following rules to calculate premium.
 - (1) If a person's health is excellent and the person is between 25 and 35 years of age and lives in a city and is a male then the premium is Rs. 4 per thousand and his policy amount cannot exceed Rs. 2 lakhs.
 - (2) If a person satisfies all the above conditions except that the sex is female then the premium is Rs. 3 per thousand and her policy amount cannot exceed Rs. 1 lakh.
 - (3) If a person's health is poor and the person is between 25 and 35 years of age and lives in a village and is a male

then the premium is Rs. 6 per thousand and his policy cannot exceed Rs. 10,000.

- (4) In all other cases the person is not insured.

Write a program to output whether the person should be insured or not, his/her premium rate and maximum amount for which he/she can be insured.

- (d) A certain grade of steel is graded according to the following conditions:
- (i) Hardness must be greater than 50
 - (ii) Carbon content must be less than 0.7
 - (iii) Tensile strength must be greater than 5600

The grades are as follows:

Grade is 10 if all three conditions are met
Grade is 9 if conditions (i) and (ii) are met
Grade is 8 if conditions (ii) and (iii) are met
Grade is 7 if conditions (i) and (iii) are met
Grade is 6 if only one condition is met
Grade is 5 if none of the conditions are met

Write a program, which will require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the steel.

- (e) A library charges a fine for every book returned late. For first 5 days the fine is 50 paise, for 6-10 days fine is one rupee and above 10 days fine is 5 rupees. If you return the book after 30 days your membership will be cancelled. Write a program to accept the number of days the member is late to return the book and display the fine or the appropriate message.

- (f) If the three sides of a triangle are entered through the keyboard, write a program to check whether the triangle is valid or not. The triangle is valid if the sum of two sides is greater than the largest of the three sides.
- (g) If the three sides of a triangle are entered through the keyboard, write a program to check whether the triangle is isosceles, equilateral, scalene or right angled triangle.
- (h) In a company, worker efficiency is determined on the basis of the time required for a worker to complete a particular job. If the time taken by the worker is between 2 – 3 hours, then the worker is said to be highly efficient. If the time required by the worker is between 3 – 4 hours, then the worker is ordered to improve speed. If the time taken is between 4 – 5 hours, the worker is given training to improve his speed, and if the time taken by the worker is more than 5 hours, then the worker has to leave the company. If the time taken by the worker is input through the keyboard, find the efficiency of the worker.
- (i) A university has the following rules for a student to qualify for a degree with A as the main subject and B as the subsidiary subject:
 - (a) He should get 55 percent or more in A and 45 percent or more in B.
 - (b) If he gets than 55 percent in A he should get 55 percent or more in B. However, he should get at least 45 percent in A.
 - (c) If he gets less than 45 percent in B and 65 percent or more in A he is allowed to reappear in an examination in B to qualify.
 - (d) In all other cases he is declared to have failed.

Write a program to receive marks in A and B and Output whether the student has passed, failed or is allowed to reappear in B.

- (j) The policy followed by a company to process customer orders is given by the following rules:
- (a) If a customer order is less than or equal to that in stock and has credit is OK, supply has requirement.
 - (b) If has credit is not OK do not supply. Send him intimation.
 - (c) If has credit is Ok but the item in stock is less than has order, supply what is in stock. Intimate to him data the balance will be shipped.

Write a C program to implement the company policy.

Conditional operators

[G] What would be the output of the following programs:

- (a)

```
main()
{
    int i = -4, j, num ;
    j = ( num < 0 ? 0 : num * num ) ;
    printf ( "\n%d", j ) ;
}
```
- (b)

```
main()
{
    int k, num = 30 ;
    k = ( num > 5 ? ( num <= 10 ? 100 : 200 ) : 500 ) ;
    printf ( "\n%d", num ) ;
}
```
- (c)

```
main()
{
    int j = 4 ;
    ( !j != 1 ? printf ( "\nWelcome" ) : printf ( "\nGood Bye" ) ) ;
}
```

```
}
```

[H] Point out the errors, if any, in the following programs:

(a) `main()`

```
{
    int tag = 0, code = 1 ;
    if ( tag == 0 )
        ( code > 1 ? printf ( "\nHello" ) ? printf ( "\nHi" ) ) ;
    else
        printf ( "\nHello Hi !!" ) ;
}
```

(b) `main()`

```
{
    int ji = 65 ;
    printf ( "\nji >= 65 ? %d : %c", ji ) ;
}
```

(c) `main()`

```
{
    int i = 10, j ;
    i >= 5 ? ( j = 10 ) : ( j = 15 ) ;
    printf ( "\n%d %d", i, j ) ;
}
```

(d) `main()`

```
{
    int a = 5 , b = 6 ;
    ( a == b ? printf( "%d",a) ) ;
}
```

(e) `main()`

```
{
    int n = 9 ;
    ( n == 9 ? printf( "You are correct" ) ; : printf( "You are wrong" ) ) ;
}
```



```
(f)  main()
    {
        int  kk = 65 ,ll ;
        ll = ( kk == 65 : printf ( "\n kk is equal to 65" ) : printf ( "\n kk is not
equal to 65" ) ) ;
        printf( "%d", ll ) ;
    }
```

```
(g)  main()
    {
        int  x = 10, y = 20 ;
        x == 20 && y != 10 ? printf( "True" ) : printf( "False" ) ;
    }
```

[I] Rewrite the following programs using conditional operators.

```
(a)  main()
    {
        int  x, min, max ;
        scanf ( "\n%d %d", &max, &x ) ;
        if ( x > max )
            max = x ;
        else
            min = x ;
    }
```

```
(b)  main()
    {
        int  code ;
        scanf ( "%d", &code ) ;
        if ( code > 1 )
            printf ( "\nJerusalem" ) ;
        else
            if ( code < 1 )
                printf ( "\nEddie" ) ;
            else
                printf ( "\nC Brain" ) ;
    }
```

```
(c)  main( )
      {
        float sal ;
        printf ("Enter the salary" ) ;
        scanf ( "%f", &sal ) ;
        if ( sal < 40000 && sal > 25000 )
          printf ( "Manager" ) ;
        else
          if ( sal < 25000 && sal > 15000 )
            printf ( "Accountant" ) ;
          else
            printf ( "Clerk" ) ;
      }
```

[J] Attempt the following:

- (a) Using conditional operators determine:
 - (1) Whether the character entered through the keyboard is a lower case alphabet or not.
 - (2) Whether a character entered through the keyboard is a special symbol or not.
- (b) Write a program using conditional operators to determine whether a year entered through the keyboard is a leap year or not.
- (c) Write a program to find the greatest of the three numbers entered through the keyboard using conditional operators.

-
- Loops
 - The *while* Loop
 - Tips and Traps
 - More Operators
 - The *for* Loop
 - Nesting of Loops
 - Multiple Initialisations in the *for* Loop
 - The Odd Loop
 - The *break* Statement
 - The *continue* Statement
 - The *do-while* Loop
 - Summary
 - Exercise

The programs that we have developed so far used either a sequential or a decision control instruction. In the first one, the calculations were carried out in a fixed order, while in the second, an appropriate set of instructions were executed depending upon the outcome of the condition being tested (or a logical decision being taken).

These programs were of limited nature, because when executed, they always performed the same series of actions, in the same way, exactly once. Almost always, if something is worth doing, it's worth doing more than once. You can probably think of several examples of this from real life, such as eating a good dinner or going for a movie. Programming is the same; we frequently need to perform an action over and over, often with variations in the details each time. The mechanism, which meets this need, is the 'loop', and loops are the subject of this chapter.

Loops

The versatility of the computer lies in its ability to perform a set of instructions repeatedly. This involves repeating some portion of the program either a specified number of times or until a particular condition is being satisfied. This repetitive operation is done through a loop control instruction.

There are three methods by way of which we can repeat a part of a program. They are:

- (a) Using a **for** statement
- (b) Using a **while** statement
- (c) Using a **do-while** statement

Each of these methods is discussed in the following pages.

The *while* Loop

It is often the case in programming that you want to do something a fixed number of times. Perhaps you want to calculate gross salaries of ten different persons, or you want to convert temperatures from centigrade to fahrenheit for 15 different cities.

The **while** loop is ideally suited for such cases. Let us look at a simple example, which uses a **while** loop. The flowchart shown below would help you to understand the operation of the **while** loop.

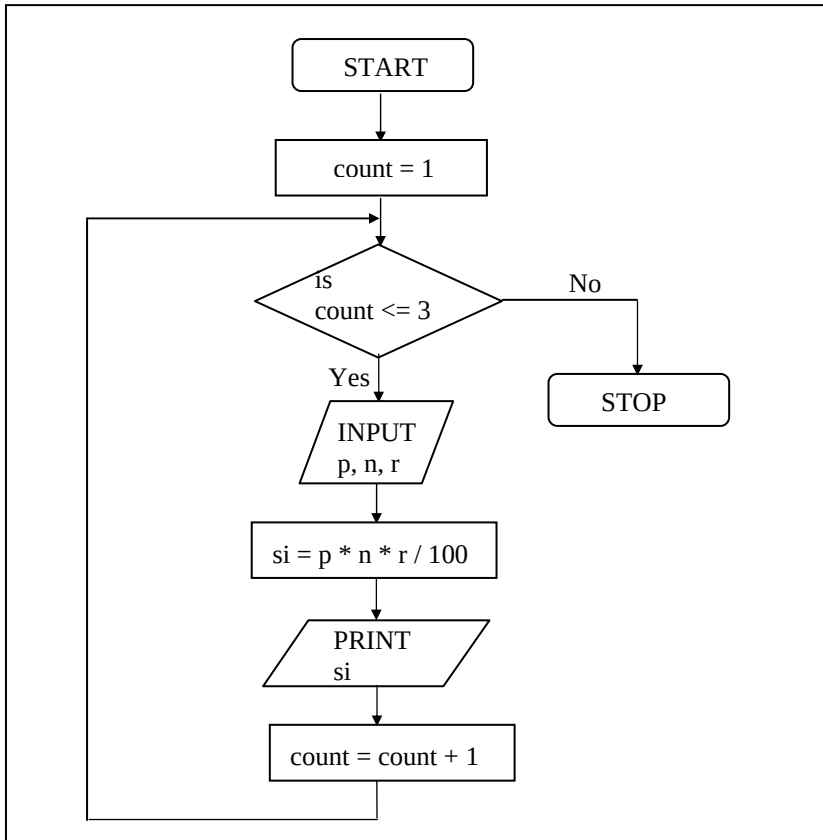


Figure 3.1

/* Calculation of simple interest for 3 sets of p, n and r */

main()

{

int p, n, count;

float r, si;

count = 1;

```
while ( count <= 3 )
{
    printf ( "\nEnter values of p, n and r " ) ;
    scanf ( "%d %d %f", &p, &n, &r ) ;
    si = p * n * r / 100 ;
    printf ( "Simple interest = Rs. %f", si ) ;

    count = count + 1 ;
}
}
```

And here are a few sample runs...

```
Enter values of p, n and r 1000 5 13.5
Simple interest = Rs. 675.000000
Enter values of p, n and r 2000 5 13.5
Simple interest = Rs. 1350.000000
Enter values of p, n and r 3500 5 3.5
Simple interest = Rs. 612.500000
```

The program executes all statements after the **while** 3 times. The logic for calculating the simple interest is written within a pair of braces immediately after the **while** keyword. These statements form what is called the ‘body’ of the **while** loop. The parentheses after the **while** contain a condition. So long as this condition remains true all statements within the body of the **while** loop keep getting executed repeatedly. To begin with the variable **count** is initialized to 1 and every time the simple interest logic is executed the value of **count** is incremented by one. The variable **count** is many a times called either a ‘loop counter’ or an ‘index variable’.

The operation of the **while** loop is illustrated in the following figure.

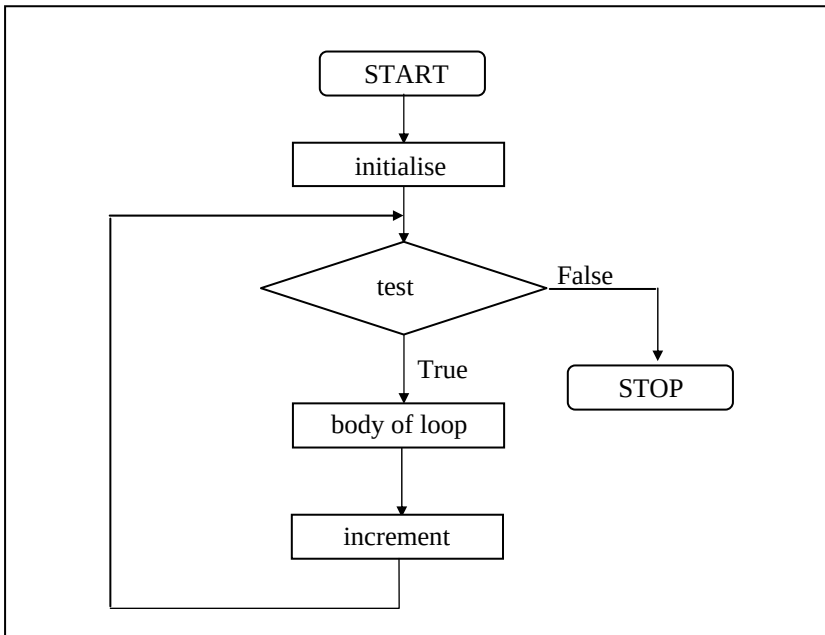


Figure 3.2

Tips and Traps

The general form of **while** is as shown below:

```
initialise loop counter ;  
while ( test loop counter using a condition )  
{  
    do this ;  
    and this ;  
    increment loop counter ;  
}
```

Note the following points about **while**...

- The statements within the **while** loop would keep on getting executed till the condition being tested remains true. When the

condition becomes false, the control passes to the first statement that follows the body of the **while** loop.

In place of the condition there can be any other valid expression. So long as the expression evaluates to a non-zero value the statements within the loop would get executed.

- The condition being tested may use relational or logical operators as shown in the following examples:

```
while ( i <= 10 )
while ( i >= 10 && j <= 15 )
while ( j > 10 && ( b < 15 || c < 20 ) )
```

- The statements within the loop may be a single line or a block of statements. In the first case the parentheses are optional. For example,

```
while ( i <= 10 )
    i = i + 1 ;
```

is same as

```
while ( i <= 10 )
{
    i = i + 1 ;
}
```

- As a rule the while must test a condition that will eventually become false, otherwise the loop would be executed forever, indefinitely.

```
main()
{
    int i = 1 ;
    while ( i <= 10 )
        printf ( "%d\n", i ) ;
}
```

This is an indefinite loop, since `i` remains equal to 1 forever. The correct form would be as under:

```
main()
{
    int i = 1;
    while (i <= 10)
    {
        printf ("%d\n", i);
        i = i + 1;
    }
}
```

- Instead of incrementing a loop counter, we can even decrement it and still manage to get the body of the loop executed repeatedly. This is shown below:

```
main()
{
    int i = 5;
    while (i >= 1)
    {
        printf ("\nMake the computer literate!");
        i = i - 1;
    }
}
```

- It is not necessary that a loop counter must only be an int. It can even be a float.

```
main()
{
    float a = 10.0;
    while (a <= 10.5)
    {
        printf ("\nRaindrops on roses...");
        printf ("...and whiskers on kittens");
        a = a + 0.1;
    }
}
```

```
    }  
}
```

- Even floating point loop counters can be decremented. Once again the increment and decrement could be by any value, not necessarily 1.

What do you think would be the output of the following program?

```
main()  
{  
    int i = 1;  
    while ( i <= 32767 )  
    {  
        printf ( "%d\n", i );  
        i = i + 1;  
    }  
}
```

No, it doesn't print numbers from 1 to 32767. It's an indefinite loop. To begin with, it prints out numbers from 1 to 32767. After that value of **i** is incremented by 1, therefore it tries to become 32768, which falls outside the valid integer range, so it goes to other side and becomes -32768 which would certainly satisfy the condition in the **while**. This process goes on indefinitely.

- What will be the output of the following program?

```
main()  
{  
    int i = 1;  
    while ( i <= 10 )  
    {  
        printf ( "%d\n", i );  
        i = i + 1;  
    }  
}
```

This is another indefinite loop, and it doesn't give any output at all. The reason is, we have carelessly given a ; after the **while**. This would make the loop work like this...

```
while ( i <= 10 )
;
{
    printf ( "%d\n", i );
    i = i + 1 ;
}
```

Since the value of **i** is not getting incremented the control would keep rotating within the loop, eternally. Note that enclosing **printf ()** and **i = i +1** within a pair of braces is not an error. In fact we can put a pair of braces around any individual statement or set of statements without affecting the execution of the program.

More Operators

There are variety of operators which are frequently used with **while**. To illustrate their usage let us consider a problem wherein numbers from 1 to 10 are to be printed on the screen. The program for performing this task can be written using **while** in the following different ways:

```
(a)  main( )
    {
        int i = 1 ;
        while ( i <= 10 )
        {
            printf ( "%d\n", i );
            i = i + 1 ;
        }
    }
```

```
(b)  main()  
    {  
        int i = 1;  
        while ( i <= 10 )  
        {  
            printf ( "%d\n", i );  
            i++;  
        }  
    }
```

Note that the increment operator `++` increments the value of `i` by 1, every time the statement `i++` gets executed. Similarly, to reduce the value of a variable by 1 a decrement operator `--` is also available.

However, never use `n+++` to increment the value of `n` by 2, since C doesn't recognize the operator `+++`.

```
(c)  main()  
    {  
        int i = 1;  
        while ( i <= 10 )  
        {  
            printf ( "%d\n", i );  
            i += 1;  
        }  
    }
```

Note that `+=` is a compound assignment operator. It increments the value of `i` by 1. Similarly, `j = j + 10` can also be written as `j += 10`. Other compound assignment operators are `-=`, `*=`, `/=` and `%=`.

```
(d)  main()  
    {  
        int i = 0;  
        while ( i++ < 10 )
```

```
    printf ( "%d\n", i );  
}
```

In the statement **while (i++ < 10)**, firstly the comparison of value of **i** with 10 is performed, and then the incrementation of **i** takes place. Since the incrementation of **i** happens after its usage, here the **++** operator is called a post-incrementation operator. When the control reaches **printf()**, **i** has already been incremented, hence **i** must be initialized to 0.

```
(e) main()  
{  
    int i = 0;  
    while ( ++i <= 10 )  
        printf ( "%d\n", i );  
}
```

In the statement **while (++i <= 10)**, firstly incrementation of **i** takes place, then the comparison of value of **i** with 10 is performed. Since the incrementation of **i** happens before its usage, here the **++** operator is called a pre-incrementation operator.

The *for* Loop

Perhaps one reason why few programmers use **while** is that they are too busy using the **for**, which is probably the most popular looping instruction. The **for** allows us to specify three things about a loop in a single line:

- (a) Setting a loop counter to an initial value.
- (b) Testing the loop counter to determine whether its value has reached the number of repetitions desired.
- (c) Increasing the value of loop counter each time the program segment within the loop has been executed.

The general form of **for** statement is as under:

```
for ( initialise counter ; test counter ; increment counter )
{
    do this ;
    and this ;
    and this ;
}
```

Let us write down the simple interest program using **for**. Compare this program with the one, which we wrote using **while**. The flowchart is also given below for a better understanding.

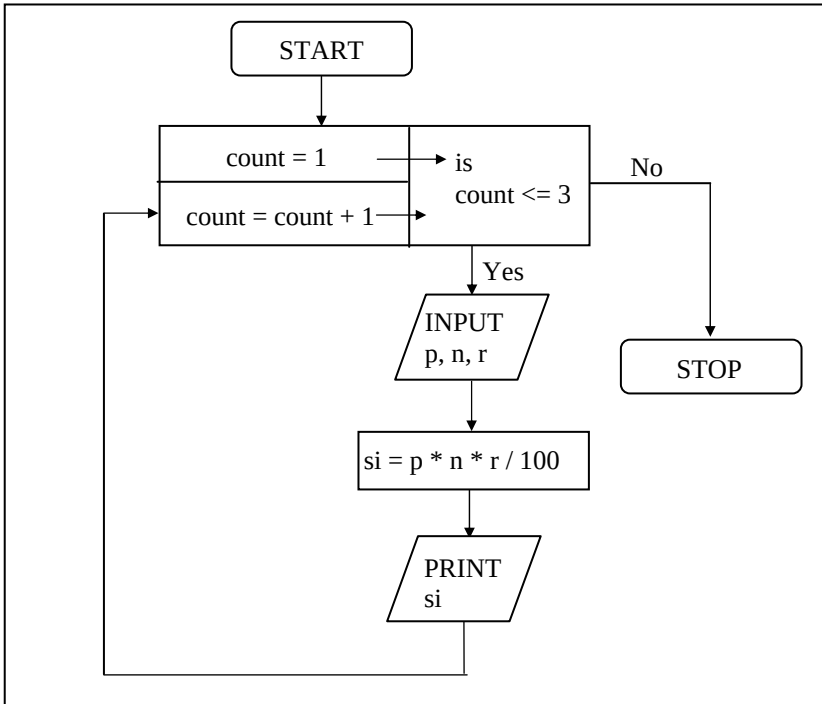


Figure 3.3

```

/* Calculation of simple interest for 3 sets of p, n and r */
main ()
{
    int  p, n, count ;
    float r, si ;

    for ( count = 1 ; count <= 3 ; count = count + 1 )
    {
        printf ( "Enter values of p, n, and r " ) ;
        scanf ( "%d %d %f", &p, &n, &r ) ;

        si = p * n * r / 100 ;
        printf ( "Simple Interest = Rs.%f\n", si ) ;
    }
}

```


If this program is compared with the one written using **while**, it can be seen that the three steps—initialization, testing and incrementation—required for the loop construct have now been incorporated in the **for** statement.

Let us now examine how the **for** statement gets executed:

- When the **for** statement is executed for the first time, the value of **count** is set to an initial value 1.
- Now the condition **count** $\leq$ 3 is tested. Since **count** is 1 the condition is satisfied and the body of the loop is executed for the first time.
- Upon reaching the closing brace of **for**, control is sent back to the **for** statement, where the value of **count** gets incremented by 1.
- Again the test is performed to check whether the new value of **count** exceeds 3.
- If the value of **count** is still within the range 1 to 3, the statements within the braces of **for** are executed again.
- The body of the **for** loop continues to get executed till **count** doesn't exceed the final value 3.
- When **count** reaches the value 4 the control exits from the loop and is transferred to the statement (if any) immediately after the body of **for**.

The following figure would help in further clarifying the concept of execution of the **for** loop.

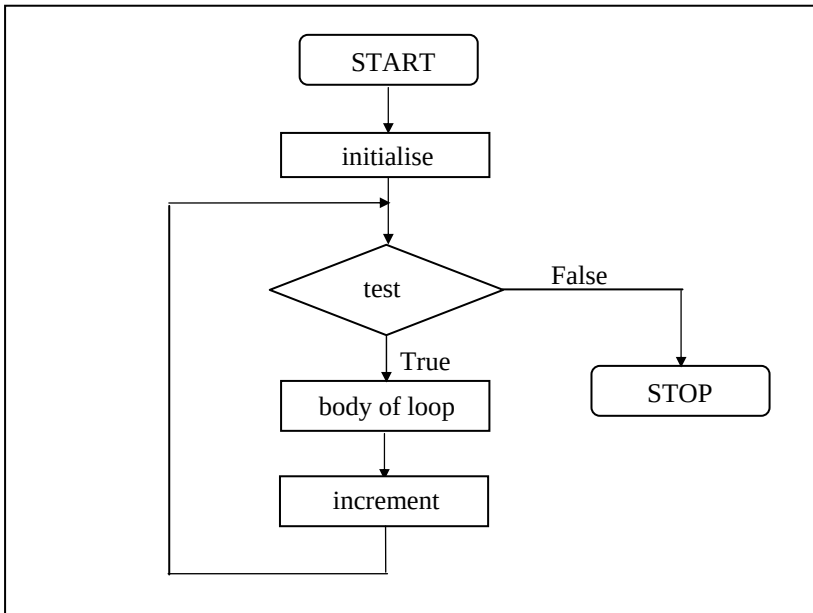


Figure 3.4

It is important to note that the initialization, testing and incrementation part of a **for** loop can be replaced by any valid expression. Thus the following **for** loops are perfectly ok.

```
for ( i = 10 ; i ; i -- )  
    printf ( "%d", i ) ;  
for ( i < 4 ; j = 5 ; j = 0 )  
    printf ( "%d", i ) ;  
for ( i = 1 ; i <= 10 ; printf ( "%d", i++ )  
    ;  
for ( scanf ( "%d", &i ) ; i <= 10 ; i++ )  
    printf ( "%d", i ) ;
```

Let us now write down the program to print numbers from 1 to 10 in different ways. This time we would use a **for** loop instead of a **while** loop.

```
(a) main()  
{  
    int i;  
    for ( i = 1 ; i <= 10 ; i = i + 1 )  
        printf ( "%d\n", i );  
}
```

Note that the initialisation, testing and incrementation of loop counter is done in the **for** statement itself. Instead of **i = i + 1**, the statements **i++** or **i += 1** can also be used.

Since there is only one statement in the body of the **for** loop, the pair of braces have been dropped. As with the **while**, the default scope of **for** is the immediately next statement after **for**.

```
(b) main()  
{  
    int i;  
    for ( i = 1 ; i <= 10 ; )  
    {  
        printf ( "%d\n", i );  
        i = i + 1 ;  
    }  
}
```

Here, the incrementation is done within the body of the **for** loop and not in the **for** statement. Note that inspite of this the semicolon after the condition is necessary.

```
(c) main()  
{  
    int i = 1 ;  
    for ( ; i <= 10 ; i = i + 1 )  
        printf ( "%d\n", i );  
}
```

Here the initialisation is done in the declaration statement itself, but still the semicolon before the condition is necessary.

```
(d) main()  
{  
    int i = 1;  
    for (; i <= 10 ; )  
    {  
        printf ("%d\n", i);  
        i = i + 1;  
    }  
}
```

Here, neither the initialisation, nor the incrementation is done in the **for** statement, but still the two semicolons are necessary.

```
(e) main()  
{  
    int i;  
    for ( i = 0 ; i++ < 10 ; )  
        printf ("%d\n", i);  
}
```

Here, the comparison as well as the incrementation is done through the same statement, **i++ < 10**. Since the ++ operator comes after **i** firstly comparison is done, followed by incrementation. Note that it is necessary to initialize **i** to 0.

```
(f) main()  
{  
    int i;  
    for ( i = 0 ; ++i <= 10 ; )  
        printf ("%d\n", i);  
}
```

Here, both, the comparison and the incrementation is done through the same statement, `++i <= 10`. Since `++` precedes `i` firstly incrementation is done, followed by comparison. Note that it is necessary to initialize `i` to 0.

Nesting of Loops

The way **if** statements can be nested, similarly **whiles** and **fors** can also be nested. To understand how nested loops work, look at the program given below:

```
/* Demonstration of nested loops */
main()
{
    int  r, c, sum ;
    for ( r = 1 ; r <= 3 ; r++ ) /* outer loop */
    {
        for ( c = 1 ; c <= 2 ; c++ ) /* inner loop */
        {
            sum = r + c ;
            printf ( "r = %d c = %d sum = %d\n", r, c, sum ) ;
        }
    }
}
```

When you run this program you will get the following output:

```
r = 1 c = 1 sum = 2
r = 1 c = 2 sum = 3
r = 2 c = 1 sum = 3
r = 2 c = 2 sum = 4
r = 3 c = 1 sum = 4
r = 3 c = 2 sum = 5
```

Here, for each value of `r` the inner loop is cycled through twice, with the variable `c` taking values from 1 to 2. The inner loop

terminates when the value of **c** exceeds 2, and the outer loop terminates when the value of **r** exceeds 3.

As you can see, the body of the outer **for** loop is indented, and the body of the inner **for** loop is further indented. These multiple indentations make the program easier to understand.

Instead of using two statements, one to calculate **sum** and another to print it out, we can compact this into one single statement by saying:

```
printf ( "r = %d c = %d sum = %d\n", r, c, r + c );
```

The way **for** loops have been nested here, similarly, two **while** loops can also be nested. Not only this, a **for** loop can occur within a **while** loop, or a **while** within a **for**.

Multiple Initialisations in the *for* Loop

The initialisation expression of the **for** loop can contain more than one statement separated by a comma. For example,

```
for ( i = 1, j = 2 ; j <= 10 ; j++ )
```

Multiple statements can also be used in the incrementation expression of **for** loop; i.e., you can increment (or decrement) two or more variables at the same time. However, only one expression is allowed in the test expression. This expression may contain several conditions linked together using logical operators.

Use of multiple statements in the initialisation expression also demonstrates why semicolons are used to separate the three expressions in the **for** loop. If commas had been used, they could not also have been used to separate multiple statements in the initialisation expression, without confusing the compiler.

The Odd Loop

The loops that we have used so far executed the statements within them a finite number of times. However, in real life programming one comes across a situation when it is not known beforehand how many times the statements in the loop are to be executed. This situation can be programmed as shown below:

```
/* Execution of a loop an unknown number of times */
main()
{
    char another ;
    int num ;
    do
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d", num, num * num ) ;
        printf ( "\nWant to enter another number y/n " ) ;
        scanf ( " %c", &another ) ;
    } while ( another == 'y' ) ;
}
```

And here is the sample output...

```
Enter a number 5
square of 5 is 25
Want to enter another number y/n y
Enter a number 7
square of 7 is 49
Want to enter another number y/n n
```

In this program the **do-while** loop would keep getting executed till the user continues to answer y. The moment he answers n, the loop terminates, since the condition (**another** == 'y') fails. Note that this loop ensures that statements within it are executed at least once even if **n** is supplied first time itself.

Though it is simpler to program such a requirement using a **do-while** loop, the same functionality if required, can also be accomplished using **for** and **while** loops as shown below:

```
/* odd loop using a for loop */
main()
{
    char another = 'y' ;
    int num ;
    for ( ; another == 'y' ; )
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d", num, num * num ) ;
        printf ( "\nWant to enter another number y/n " ) ;
        scanf ( " %c", &another ) ;
    }
}
```

```
/* odd loop using a while loop */
main()
{
    char another = 'y' ;
    int num ;

    while ( another == 'y' )
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d", num, num * num ) ;
        printf ( "\nWant to enter another number y/n " ) ;
        scanf ( " %c", &another ) ;
    }
}
```


The *break* Statement

We often come across situations where we want to jump out of a loop instantly, without waiting to get back to the conditional test. The keyword **break** allows us to do this. When **break** is encountered inside any loop, control automatically passes to the first statement after the loop. A **break** is usually associated with an **if**. As an example, let's consider the following example.

Example: Write a program to determine whether a number is prime or not. A prime number is one, which is divisible only by 1 or itself.

All we have to do to test whether a number is prime or not, is to divide it successively by all numbers from 2 to one less than itself. If remainder of any of these divisions is zero, the number is not a prime. If no division yields a zero then the number is a prime number. Following program implements this logic.

```
main()
{
    int num, i ;

    printf ( "Enter a number " ) ;
    scanf ( "%d", &num ) ;

    i = 2 ;
    while ( i <= num - 1 )
    {
        if ( num % i == 0 )
        {
            printf ( "Not a prime number" ) ;
            break ;
        }
        i++ ;
    }
}
```

```
    if ( i == num )
        printf ( "Prime number" ) ;
}
```

In this program the moment **num % i** turns out to be zero, (i.e. **num** is exactly divisible by **i**) the message “Not a prime number” is printed and the control breaks out of the **while** loop. Why does the program require the **if** statement after the **while** loop at all? Well, there are two ways the control could have reached outside the **while** loop:

- (a) It jumped out because the number proved to be not a prime.
- (b) The loop came to an end because the value of **i** became equal to **num**.

When the loop terminates in the second case, it means that there was no number between 2 to **num - 1** that could exactly divide **num**. That is, **num** is indeed a prime. If this is true, the program should print out the message “Prime number”.

The keyword **break**, breaks the control only from the **while** in which it is placed. Consider the following program, which illustrates this fact.

```
main()
{
    int i = 1 , j = 1 ;

    while ( i++ <= 100 )
    {
        while ( j++ <= 200 )
        {
            if ( j == 150 )
                break ;
            else
                printf ( "%d %d\n", i, j ) ;
        }
    }
```

```
    }  
}
```

In this program when **j** equals 150, **break** takes the control outside the inner **while** only, since it is placed inside the inner **while**.

The *continue* Statement

In some programming situations we want to take the control to the beginning of the loop, bypassing the statements inside the loop, which have not yet been executed. The keyword **continue** allows us to do this. When **continue** is encountered inside any loop, control automatically passes to the beginning of the loop.

A **continue** is usually associated with an **if**. As an example, let's consider the following program.

```
main()  
{  
    int i, j;  
  
    for ( i = 1 ; i <= 2 ; i++ )  
    {  
        for ( j = 1 ; j <= 2 ; j++ )  
        {  
            if ( i == j )  
                continue ;  
  
            printf ( "\n%d %d\n", i, j ) ;  
        }  
    }  
}
```

The output of the above program would be...

```
1 2  
2 1
```

Note that when the value of **i** equals that of **j**, the **continue** statement takes the control to the **for** loop (inner) bypassing rest of the statements pending execution in the **for** loop (inner).

The *do-while* Loop

The **do-while** loop looks like this:

```
do
{
    this ;
    and this ;
    and this ;
    and this ;
} while ( this condition is true ) ;
```

There is a minor difference between the working of **while** and **do-while** loops. This difference is the place where the condition is tested. The **while** tests the condition before executing any of the statements within the **while** loop. As against this, the **do-while** tests the condition after having executed the statements within the loop. Figure 3.5 would clarify the execution of **do-while** loop still further.

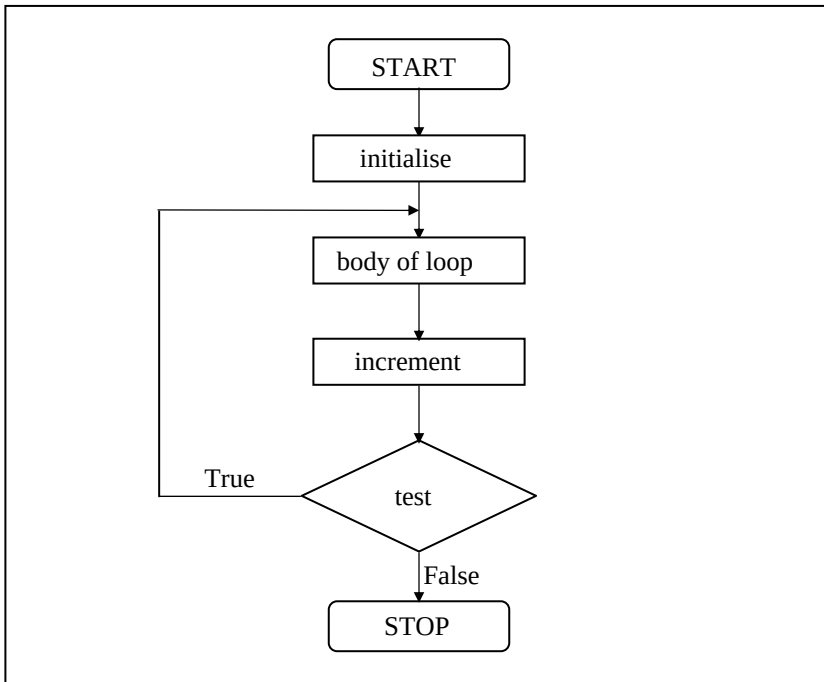


Figure 3.5

This means that **do-while** would execute its statements at least once, even if the condition fails for the first time. The **while**, on the other hand will not execute its statements if the condition fails for the first time. This difference is brought about more clearly by the following program.

```
main()  
{  
    while ( 4 < 1 )  
        printf ( "Hello there \n" );  
}
```

Here, since the condition fails the first time itself, the **printf()** will not get executed at all. Let's now write the same program using a **do-while** loop.

```
main()  
{  
    do  
    {  
        printf ( "Hello there \n" );  
    } while ( 4 < 1 );  
}
```

In this program the **printf()** would be executed once, since first the body of the loop is executed and then the condition is tested.

There are some occasions when we want to execute a loop at least once no matter what. This is illustrated in the following example:

break and **continue** are used with **do-while** just as they would be in a **while** or a **for** loop. A **break** takes you out of the **do-while** bypassing the conditional test. A **continue** sends you straight to the test at the end of the loop.

Summary

- (a) The three type of loops available in C are **for**, **while**, and **do-while**.
- (b) A **break** statement takes the execution control out of the loop.
- (c) A **continue** statement skips the execution of the statements after it and takes the control to the beginning of the loop.
- (d) A **do-while** loop is used to ensure that the statements within the loop are executed at least once.
- (e) The ++ operator increments the operand by 1, whereas, the -- operator decrements it by 1.
- (f) The operators +=, -=, *=, /=, %= are compound assignment operators. They modify the value of the operand to the left of them.

Exercise

while Loop

[A] What would be the output of the following programs:

```
(a)  main()  
    {  
        int j;  
        while ( j <= 10 )  
        {  
            printf ( "\n%d", j );  
            j = j + 1;  
        }  
    }
```

```
(b)  main()  
    {  
        int i = 1;  
        while ( i <= 10 )  
        {  
            printf ( "\n%d", i );  
        }
```

```
        i++ ;  
    }  
}
```

```
(c)  main()  
    {  
        int j ;  
        while ( j <= 10 )  
        {  
            printf ( "\n%d", j ) ;  
            j = j + 1 ;  
        }  
    }
```

```
(d)  main()  
    {  
        int x = 1 ;  
        while ( x == 1 )  
        {  
            x = x - 1 ;  
            printf ( "\n%d", x ) ;  
        }  
    }
```

```
(e)  main()  
    {  
        int x = 1 ;  
        while ( x == 1 )  
            x = x - 1 ;  
        printf ( "\n%d", x ) ;  
    }
```

```
(f)  main()  
    {  
        char x ;
```



```
        while ( x = 0 ; x <= 255 ; x++ )
            printf ( "\nAscii value %d Character %c", x, x ) ;
    }

(g)  main( )
    {
        int x = 4, y, z ;
        y = --x ;
        z = x-- ;
        printf ( "\n%d %d %d", x, y, z ) ;
    }

(h)  main( )
    {
        int x = 4, y = 3, z ;

        z = x-- -y ;
        printf ( "\n%d %d %d", x, y, z ) ;
    }

(i)  main( )
    {
        while ( 'a' < 'b' )
            printf ( "\nmalyalam is a palindrome" ) ;
    }

(j)  main( )
    {
        int i = 10 ;
        while ( i = 20 )
            printf ( "\nA computer buff!" ) ;
    }

(k)  main( )
    {
        int i ;
        while ( i = 10 )
        {
```

```
        printf ( "\n%d", i );
        i = i + 1 ;
    }
}
```

```
(l)  main()
    {
        float x = 1.1 ;
        while ( x == 1.1 )
        {
            printf ( "\n%f", x );
            x = x - 0.1 ;
        }
    }
```

```
(m)  main()
    {
        while ( '1' < '2' )
            printf ( "\n\n while loop" ) ;
    }
```

```
(n)  main()
    {
        char x ;
        for ( x = 0 ; x <= 255 ; x++ )
            printf ( "\nAscii value %d Character %c", x, x ) ;
    }
```

```
(o)  main()
    {
        int x = 4, y = 0, z ;
        while ( x >= 0 )
        {
            x-- ;
            y++ ;
            if ( x == y )
```

```
        continue ;
    else
        printf ( "\n%d %d", x, y ) ;
    }
}
```

(p) main()

```
{
    int x = 4, y = 0, z ;
    while ( x >= 0 )
    {
        if ( x == y )
            break ;
        else
            printf ( "\n%d %d", x, y ) ;
        x-- ;
        y++ ;
    }
}
```

[B] Attempt the following:

- (a) Write a program to calculate overtime pay of 10 employees. Overtime is paid at the rate of Rs. 12.00 per hour for every hour worked above 40 hours. Assume that employees do not work for fractional part of an hour.
- (b) Write a program to find the factorial value of any number entered through the keyboard.
- (c) Two numbers are entered through the keyboard. Write a program to find the value of one number raised to the power of another.
- (d) Write a program to print all the ASCII values and their equivalent characters using a **while** loop. The ASCII values vary from 0 to 255.

- (e) Write a program to print out all Armstrong numbers between 1 and 500. If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number. For example, $153 = (1 * 1 * 1) + (5 * 5 * 5) + (3 * 3 * 3)$
- (f) Write a program for a matchstick game being played between the computer and a user. Your program should ensure that the computer always wins. Rules for the game are as follows:
- There are 21 matchsticks.
 - The computer asks the player to pick 1, 2, 3, or 4 matchsticks.
 - After the person picks, the computer does its picking.
 - Whoever is forced to pick up the last matchstick loses the game.
- (g) Write a program to enter the numbers till the user wants and at the end it should display the count of positive, negative and zeros entered.
- (h) Write a program to find the octal equivalent of the entered number.
- (i) Write a program to find the range of a set of numbers. Range is the difference between the smallest and biggest number in the list.

for, break, continue, do-while

[C] What would be the output of the following programs:

- (a)

```
main()  
{  
    int i = 0;  
    for (; i ; )
```

```
        printf ( "\nHere is some mail for you" );
    }
```

```
(b)  main()
    {
        int i ;
        for ( i = 1 ; i <= 5 ; printf ( "\n%d", i ) ) ;
            i++ ;
    }
```

```
(c)  main()
    {
        int i = 1, j = 1 ;
        for ( ; ; )
        {
            if ( i > 5 )
                break ;
            else
                j += i ;
            printf ( "\n%d", j ) ;
            i += j ;
        }
    }
```

```
(d)  main()
    {
        int i ;
        for ( i = 1 ; i <= 5 ; printf ( "\n%c", 65 ) ) ;
            i++ ;
    }
```

[D] Answer the following:

(a) The three parts of the loop expression in the **for** loop are:

the expression
 the expression
 the expression

- (b) An expression contains relational operators, assignment operators, and arithmetic operators. In the absence of parentheses, they will be evaluated in which of the following order:
1. assignment, relational, arithmetic
 2. arithmetic, relational, assignment
 3. relational, arithmetic, assignment
 4. assignment, arithmetic, relational
- (c) The **break** statement is used to exit from:
1. an **if** statement
 2. a **for** loop
 3. a program
 4. the **main()** function
- (d) A **do-while** loop is useful when we want that the statements within the loop must be executed:
1. Only once
 2. At least once
 3. More than once
 4. None of the above
- (e) In what sequence the initialization, testing and execution of body is done in a do-while loop
1. Initialization, execution of body, testing
 2. Execution of body, initialization, testing
 3. Initialization, testing, execution of body
 4. None of the above
- (f) Which of the following is not an infinite loop.
1.

```
int i = 1 ;  
while ( 1 )  
{  
    i++ ;  
}
```
 2.

```
for ( ; ; )
```

```
3. int True = 0, false ;
   while ( True )
   {
       False = 1 ;
   }
```

```
4. int y, x = 0 ;
   do
   {
       y = x ;
   } while ( x == 0 ) ;
```

(g) Which of the following statement is used to take the control to the beginning of the loop?

1. exit
2. break
3. continue
4. None of the above

[E] Attempt the following:

(a) Write a program to print all prime numbers from 1 to 300.
(Hint: Use nested loops, **break** and **continue**)

(b) Write a program to fill the entire screen with a smiling face.
The smiling face has an ASCII value 1.

(c) Write a program to add first seven terms of the following series using a **for** loop:

$$\frac{1}{1!} + \frac{2}{2!} + \frac{3}{3!} + \dots$$

(d) Write a program to generate all combinations of 1, 2 and 3 using **for** loop.

(e) According to a study, the approximate level of intelligence of a person can be calculated using the following formula:

$$i = 2 + (y + 0.5x)$$

Write a program, which will produce a table of values of **i**, **y** and **x**, where **y** varies from 1 to 6, and, for each value of **y**, **x** varies from 5.5 to 12.5 in steps of 0.5.

- (f) Write a program to produce the following output:

```

A B C D E F G F E D C B A
A B C D E F      F E D C B A
A B C D E          E D C B A
A B C D              D C B A
A B C                  C B A
A B                      B A
A                          A

```

- (g) Write a program to fill the entire screen with diamond and heart alternatively. The ASCII value for heart is 3 and that of diamond is 4.

- (h) Write a program to print the multiplication table of the number entered by the user. The table should get displayed in the following form.

```

29 * 1 = 29
29 * 2 = 58
...

```

- (i) Write a program to produce the following output:

```

          1
        2   3
      4     5   6
    7       8   9   10

```


- (j) Write a program to produce the following output:

```

      1
    1  1
  1  2  1
1  3  3  1
  4  6  4  1

```

- (k) A machine is purchased which will produce earning of Rs. 1000 per year while it lasts. The machine costs Rs. 6000 and will have a salvage of Rs. 2000 when it is condemned. If 12 percent per annum can be earned on alternate investments what would be the minimum life of the machine to make it a more attractive investment compared to alternative investment?
- (l) When interest compounds **q** times per year at an annual rate of **r** % for **n** years, the principle **p** compounds to an amount **a** as per the following formula

$$a = p (1 + r / q)^{nq}$$

Write a program to read 10 sets of **p**, **r**, **n** & **q** and calculate the corresponding **a**.

- (m) The natural logarithm can be approximated by the following series.

$$\frac{x-1}{x} + \frac{1}{2} \left(\frac{x-1}{x} \right)^2 + \frac{1}{2} \left(\frac{x-1}{x} \right)^3 + \frac{1}{2} \left(\frac{x-1}{x} \right)^4 + \dots$$

If **x** is input through the keyboard, write a program to calculate the sum of first seven terms of this series.

4 *The Case Control Structure*

- Decisions Using *switch*
 The Tips and Traps
- *switch* Versus *if-else* Ladder
- The *goto* Keyword
- Summary
- Exercise

In real life we are often faced with situations where we are required to make a choice between a number of alternatives rather than only one or two. For example, which school to join or which hotel to visit or still harder which girl to marry (you almost always end up making a wrong decision is a different matter altogether!). Serious C programming is same; the choice we are asked to make is more complicated than merely selecting between two alternatives. C provides a special control statement that allows us to handle such cases effectively; rather than using a series of **if** statements. This control instruction is in fact the topic of this chapter. Towards the end of the chapter we would also study a keyword called **goto**, and understand why we should avoid its usage in C programming.

Decisions Using *switch*

The control statement that allows us to make a decision from the number of choices is called a **switch**, or more correctly a **switch-case-default**, since these three keywords go together to make up the control statement. They most often appear as follows:

```
switch ( integer expression )
{
    case constant 1 :
        do this ;
    case constant 2 :
        do this ;
    case constant 3 :
        do this ;
    default :
        do this ;
}
```

The integer expression following the keyword **switch** is any C expression that will yield an integer value. It could be an integer constant like 1, 2 or 3, or an expression that evaluates to an

integer. The keyword **case** is followed by an integer or a character constant. Each constant in each **case** must be different from all the others. The “do this” lines in the above form of **switch** represent any valid C statement.

What happens when we run a program containing a **switch**? First, the integer expression following the keyword **switch** is evaluated. The value it gives is then matched, one by one, against the constant values that follow the **case** statements. When a match is found, the program executes the statements following that **case**, and all subsequent **case** and **default** statements as well. If no match is found with any of the **case** statements, only the statements following the **default** are executed. A few examples will show how this control structure works.

Consider the following program:

```
main()
{
    int i = 2 ;

    switch ( i )
    {
        case 1 :
            printf ( "I am in case 1 \n" ) ;
        case 2 :
            printf ( "I am in case 2 \n" ) ;
        case 3 :
            printf ( "I am in case 3 \n" ) ;
        default :
            printf ( "I am in default \n" ) ;
    }
}
```

The output of this program would be:

I am in case 2

```
I am in case 3
I am in default
```

The output is definitely not what we expected! We didn't expect the second and third line in the above output. The program prints case 2 and 3 and the default case. Well, yes. We said the **switch** executes the case where a match is found and all the subsequent **cases** and the **default** as well.

If you want that only case 2 should get executed, it is upto you to get out of the **switch** then and there by using a **break** statement. The following example shows how this is done. Note that there is no need for a **break** statement after the **default**, since the control comes out of the **switch** anyway.

```
main( )
{
    int i = 2 ;

    switch ( i )
    {
        case 1 :
            printf ( "I am in case 1 \n" ) ;
            break ;
        case 2 :
            printf ( "I am in case 2 \n" ) ;
            break ;
        case 3 :
            printf ( "I am in case 3 \n" ) ;
            break ;
        default :
            printf ( "I am in default \n" ) ;
    }
}
```

The output of this program would be:

```
I am in case 2
```

The operation of **switch** is shown below in the form of a flowchart for a better understanding.

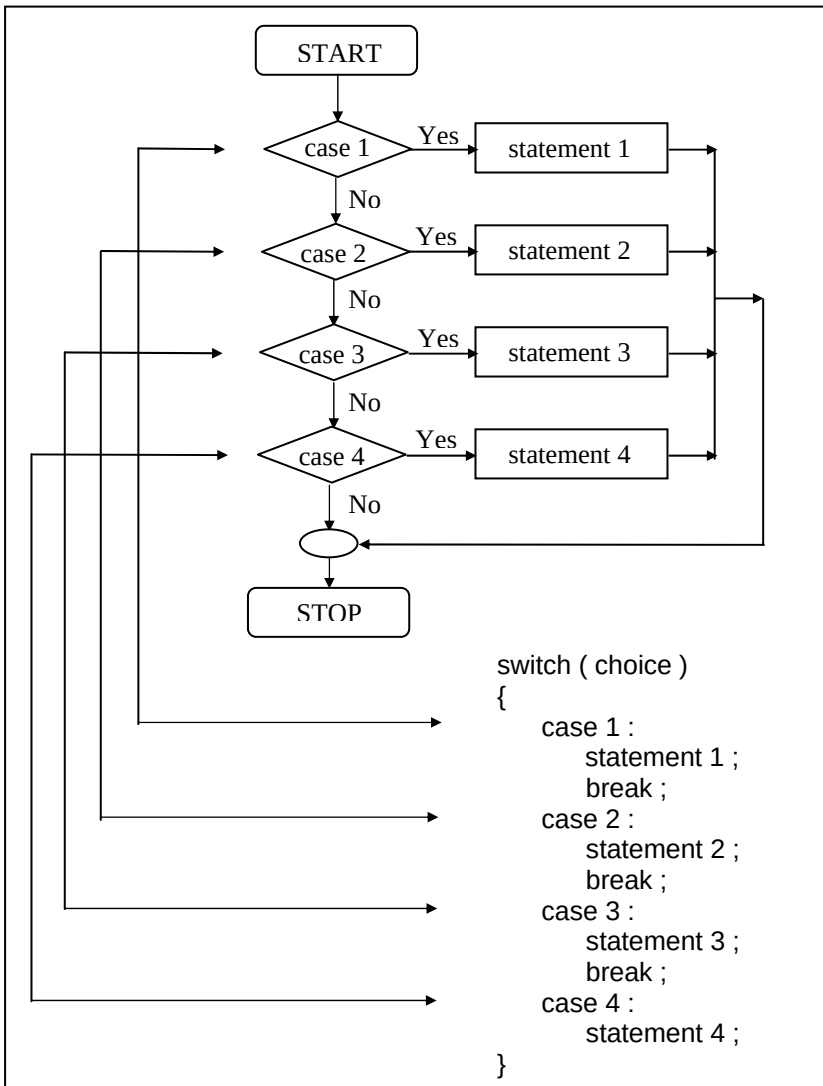


Figure 4.1

The Tips and Traps

A few useful tips about the usage of **switch** and a few pitfalls to be avoided:

- (a) The earlier program that used **switch** may give you the wrong impression that you can use only cases arranged in ascending order, 1, 2, 3 and default. You can in fact put the cases in any order you please. Here is an example of scrambled case order:

```
main()  
{  
    int i = 22 ;  
  
    switch ( i )  
    {  
        case 121 :  
            printf ( "I am in case 121 \n" ) ;  
            break ;  
        case 7 :  
            printf ( "I am in case 7 \n" ) ;  
            break ;  
        case 22 :  
            printf ( "I am in case 22 \n" ) ;  
            break ;  
        default :  
            printf ( "I am in default \n" ) ;  
    }  
}
```

The output of this program would be:

I am in case 22

- (b) You are also allowed to use **char** values in **case** and **switch** as shown in the following program:

```
main()
```

```
{
    char c = 'x' ;

    switch ( c )
    {
        case 'v' :
            printf ( "I am in case v \n" ) ;
            break ;
        case 'a' :
            printf ( "I am in case a \n" ) ;
            break ;
        case 'x' :
            printf ( "I am in case x \n" ) ;
            break ;
        default :
            printf ( "I am in default \n" ) ;
    }
}
```

The output of this program would be:

I am in case x

In fact here when we use 'v', 'a', 'x' they are actually replaced by the ASCII values (118, 97, 120) of these character constants.

- (c) At times we may want to execute a common set of statements for multiple **cases**. How this can be done is shown in the following example.

```
main()
{
    char ch ;

    printf ( "Enter any of the alphabet a, b, or c " ) ;
    scanf ( "%c", &ch ) ;
```



```
switch ( ch )
{
    case 'a' :
    case 'A' :
        printf ( "a as in ashar" ) ;
        break ;
    case 'b' :
    case 'B' :
        printf ( "b as in brain" ) ;
        break ;
    case 'c' :
    case 'C' :
        printf ( "c as in cookie" ) ;
        break ;
    default :
        printf ( "wish you knew what are alphabets" ) ;
}
```

Here, we are making use of the fact that once a **case** is satisfied the control simply falls through the **case** till it doesn't encounter a **break** statement. That is why if an alphabet **a** is entered the **case 'a'** is satisfied and since there are no statements to be executed in this **case** the control automatically reaches the next **case** i.e. **case 'A'** and executes all the statements in this **case**.

- (d) Even if there are multiple statements to be executed in each **case** there is no need to enclose them within a pair of braces (unlike **if**, and **else**).
- (e) Every statement in a **switch** must belong to some **case** or the other. If a statement doesn't belong to any **case** the compiler won't report an error. However, the statement would never get executed. For example, in the following program the **printf()** never goes to work.

```
main()
{
    int i, j;

    printf ( "Enter value of i" );
    scanf ( "%d", &i );

    switch ( i )
    {
        printf ( "Hello" );
        case 1 :
            j = 10 ;
            break ;
        case 2 :
            j = 20 ;
            break ;
    }
}
```

- (f) If we have no **default** case, then the program simply falls through the entire **switch** and continues with the next instruction (if any,) that follows the closing brace of **switch**.
- (g) Is **switch** a replacement for **if**? Yes and no. Yes, because it offers a better way of writing programs as compared to **if**, and no because in certain situations we are left with no choice but to use **if**. The disadvantage of **switch** is that one cannot have a case in a **switch** which looks like:

```
case i <= 20 :
```

All that we can have after the case is an **int** constant or a **char** constant or an expression that evaluates to one of these constants. Even a **float** is not allowed.

The advantage of **switch** over **if** is that it leads to a more structured program and the level of indentation is manageable,

more so if there are multiple statements within each **case** of a **switch**.

- (h) We can check the value of any expression in a **switch**. Thus the following **switch** statements are legal.

```
switch ( i + j * k )  
switch ( 23 + 45 % 4 * k )  
switch ( a < 4 && b > 7 )
```

Expressions can also be used in cases provided they are constant expressions. Thus **case 3 + 7** is correct, however, **case a + b** is incorrect.

- (i) The **break** statement when used in a **switch** takes the control outside the **switch**. However, use of **continue** will not take the control to the beginning of **switch** as one is likely to believe.
- (j) In principle, a **switch** may occur within another, but in practice it is rarely done. Such statements would be called nested **switch** statements.
- (k) The **switch** statement is very useful while writing menu driven programs. This aspect of **switch** is discussed in the exercise at the end of this chapter.

***switch* Versus if-else Ladder**

There are some things that you simply cannot do with a **switch**. These are:

- (a) A float expression cannot be tested using a **switch**
- (b) Cases can never have variable expressions (for example it is wrong to say **case a + 3 :**)
- (c) Multiple cases cannot use same expressions. Thus the following **switch** is illegal:

```
switch ( a )
{
    case 3 :
        ...
    case 1 + 2 :
        ...
}
```

(a), (b) and (c) above may lead you to believe that these are obvious disadvantages with a **switch**, especially since there weren't any such limitations with **if-else**. Then why use a **switch** at all? For speed—**switch** works faster than an equivalent **if-else** ladder. How come? This is because the compiler generates a jump table for a **switch** during compilation. As a result, during execution it simply refers the jump table to decide which case should be executed, rather than actually checking which case is satisfied. As against this, **if-elses** are slower because they are evaluated at execution time. A **switch** with 10 cases would work faster than an equivalent **if-else** ladder. Also, a **switch** with 2 cases would work slower than **if-else** ladder. Why? If the 10th case is satisfied then jump table would be referred and statements for the 10th case would be executed. As against this, in an **if-else** ladder 10 conditions would be evaluated at execution time, which makes it slow. Note that a lookup in the jump table is faster than evaluation of a condition, especially if the condition is complex.

If on the other hand the conditions in the **if-else** were simple and less in number then **if-else** would work out faster than the lookup mechanism of a **switch**. Hence a **switch** with two cases would work slower than an equivalent **if-else**. Thus, you as a programmer should take a decision which of the two should be used when.

The *goto* Keyword

Avoid **goto** keyword! They make a C programmer's life miserable. There is seldom a legitimate reason for using **goto**, and its use is

one of the reasons that programs become unreliable, unreadable, and hard to debug. And yet many programmers find **goto** seductive.

In a difficult programming situation it seems so easy to use a **goto** to take the control where you want. However, almost always, there is a more elegant way of writing the same program using **if**, **for**, **while** and **switch**. These constructs are far more logical and easy to understand.

The big problem with **gotos** is that when we do use them we can never be sure how we got to a certain point in our code. They obscure the flow of control. So as far as possible skip them. You can always get the job done without them. Trust me, with good programming skills **goto** can always be avoided. This is the first and last time that we are going to use **goto** in this book. However, for sake of completeness of the book, the following program shows how to use **goto**.

```
main()
{
    int goals ;

    printf ( "Enter the number of goals scored against India" ) ;
    scanf ( "%d", &goals ) ;

    if ( goals <= 5 )
        goto sos ;
    else
    {
        printf ( "About time soccer players learnt C\n" ) ;
        printf ( "and said goodbye! adieu! to soccer" ) ;
        exit( ) ; /* terminates program execution */
    }

    sos :
        printf ( "To err is human!" ) ;
```

```
}
```

And here are two sample runs of the program...

```
Enter the number of goals scored against India 3
To err is human!
Enter the number of goals scored against India 7
About time soccer players learnt C
and said goodbye! adieu! to soccer
```

A few remarks about the program would make the things clearer.

- If the condition is satisfied the **goto** statement transfers control to the label 'sos', causing **printf()** following **sos** to be executed.
- The label can be on a separate line or on the same line as the statement following it, as in,

```
sos : printf ( "To err is human!" );
```

- Any number of **gotos** can take the control to the same label.
- The **exit()** function is a standard library function which terminates the execution of the program. It is necessary to use this function since we don't want the statement

```
printf ( "To err is human!" )
```

to get executed after execution of the **else** block.

- The only programming situation in favour of using **goto** is when we want to take the control out of the loop that is contained in several other loops. The following program illustrates this.

```
main()
{
    int i, j, k ;

    for ( i = 1 ; i <= 3 ; i++ )
    {
        for ( j = 1 ; j <= 3 ; j++ )
        {
            for ( k = 1 ; k <= 3 ; k++ )
            {
                if ( i == 3 && j == 3 && k == 3 )
                    goto out ;
                else
                    printf ( "%d %d %d\n", i, j, k ) ;
            }
        }
    }
    out :
        printf ( "Out of the loop at last!" ) ;
}
```

Go through the program carefully and find out how it works. Also write down the same program without using **goto**.

Summary

- (a) When we need to choose one among number of alternatives, a **switch** statement is used.
- (b) The **switch** keyword is followed by an integer or an expression that evaluates to an integer.
- (c) The **case** keyword is followed by an integer or a character constant.
- (d) The control falls through all the cases unless the **break** statement is given.
- (e) The usage of the **goto** keyword should be avoided as it usually violets the normal flow of execution.

Exercise

[A] What would be the output of the following programs:

```
(a)  main( )
      {
        char suite = 3 ;
        switch ( suite )
        {
          case 1 :
            printf ( "\nDiamond" ) ;
          case 2 :
            printf ( "\nSpade" ) ;
          default :
            printf ( "\nHeart" ) ;
        }
        printf ( "\nI thought one wears a suite" ) ;
      }
```

```
(b)  main( )
      {
        int c = 3 ;

        switch ( c )
        {
          case 'v' :
            printf ( "I am in case v \n" ) ;
            break ;
          case 3 :
            printf ( "I am in case 3 \n" ) ;
            break ;
          case 12 :
            printf ( "I am in case 12 \n" ) ;
            break ;
          default :
            printf ( "I am in default \n" ) ;
        }
      }
```



```
}
```

```
(c) main()  
{  
    int k, j = 2 ;  
    switch ( k = j + 1 )  
    {  
        case 0 :  
            printf ( "\nTailor" ) ;  
        case 1 :  
            printf ( "\nTutor" ) ;  
        case 2 :  
            printf ( "\nTramp" ) ;  
        default :  
            printf ( "\nPure Simple Egghead!" ) ;  
    }  
}
```

```
(d) main()  
{  
    int i = 0 ;  
    switch ( i )  
    {  
        case 0 :  
            printf ( "\nCustomers are dicey" ) ;  
        case 1 :  
            printf ( "\nMarkets are pricey" ) ;  
        case 2 :  
            printf ( "\nInvestors are moody" ) ;  
        case 3 :  
            printf ( "\nAt least employees are good" ) ;  
    }  
}
```

```
(e) main()  
{  
    int k ;  
    float j = 2.0 ;
```

```
        switch ( k = j + 1 )
        {
            case 3 :
                printf ( "\nTrapped" );
                break ;
            default :
                printf ( "\nCaught!" );
        }
    }

(f)  main( )
    {
        int ch = 'a' + 'b' ;
        switch ( ch )
        {
            case 'a' :
            case 'b' :
                printf ( "\nYou entered b" );
            case 'A' :
                printf ( "\na as in ashar" );
            case 'b' + 'a' :
                printf ( "\nYou entered a and b" );
        }
    }

(g)  main( )
    {
        int i = 1 ;
        switch ( i - 2 )
        {
            case -1 :
                printf ( "\nFeeding fish" );
            case 0 :
                printf ( "\nWeeding grass" );
            case 1 :
                printf ( "\nmending roof" );
            default :
                printf ( "\nJust to survive" );
        }
    }
```

```
    }  
}
```

[B] Point out the errors, if any, in the following programs:

(a) `main()`

```
{  
    int suite = 1 ;  
    switch ( suite ) ;  
    {  
        case 0 ;  
            printf ( "\nClub" ) ;  
        case 1 ;  
            printf ( "\nDiamond" ) ;  
    }  
}
```

(b) `main()`

```
{  
    int temp ;  
    scanf ( "%d", &temp ) ;  
    switch ( temp )  
    {  
        case ( temp <= 20 ) :  
            printf ( "\nOooooooooohhhh! Damn cool!" ) ;  
        case ( temp > 20 && temp <= 30 ) :  
            printf ( "\nRain rain here again!" ) ;  
        case ( temp > 30 && temp <= 40 ) :  
            printf ( "\nWish I am on Everest" ) ;  
        default :  
            printf ( "\nGood old nagpur weather" ) ;  
    }  
}
```

(c) `main()`

```
{  
    float a = 3.5 ;  
    switch ( a )
```

```
{
    case 0.5 :
        printf ( "\nThe art of C" );
        break ;
    case 1.5 :
        printf ( "\nThe spirit of C" );
        break ;
    case 2.5 :
        printf ( "\nSee through C" );
        break ;
    case 3.5 :
        printf ( "\nSimply c" );
}
}
```

(d) main()

```
{
    int a = 3, b = 4, c ;
    c = b - a ;
    switch ( c )
    {
        case 1 || 2 :
            printf ( "God give me an opportunity to change things" );
            break ;

        case a || b :
            printf ( "God give me an opportunity to run my show" );
            break ;
    }
}
```

[C] Write a menu driven program which has following options:

1. Factorial of a number.
2. Prime or not
3. Odd or even
4. Exit

Make use of *switch* statement.

The outline of this program is given below:

```
/* A menu driven program */
main()
{
    int choice ;
    while ( 1 )
    {
        printf ( "\n1. Factorial" ) ;
        printf ( "\n2. Prime" ) ;
        printf ( "\n3. Odd/Even" ) ;
        printf ( "\n4. Exit" ) ;
        printf ( "\nYour choice? " ) ;
        scanf ( "%d", &choice ) ;

        switch ( choice )
        {
            case 1 :
                /* logic for factorial of a number */
                break ;
            case 2 :
                /* logic for deciding prime number */
                break ;
            case 3 :
                /* logic for odd/even */
                break ;
            case 4 :
                exit( ) ;
        }
    }
}
```

Note:

The statement **while (1)** puts the entire logic in an infinite loop. This is necessary since the menu must keep reappearing on the screen once an item is selected and an appropriate action taken.

[D] Write a program which to find the grace marks for a student using **switch**. The user should enter the class obtained by the student and the number of subjects he has failed in.

- If the student gets first class and the number of subjects he failed in is greater than 3, then he does not get any grace. If the number of subjects he failed in is less than or equal to 3 then the grace is of 5 marks per subject.
- If the student gets second class and the number of subjects he failed in is greater than 2, then he does not get any grace. If the number of subjects he failed in is less than or equal to 2 then the grace is of 4 marks per subject.
- If the student gets third class and the number of subjects he failed in is greater than 1, then he does not get any grace. If the number of subjects he failed in is equal to 1 then the grace is of 5 marks per subject

5 *Functions & Pointers*

- What is a Function
 - Why Use Functions
- Passing Values between Functions
- Scope Rule of Functions
- Calling Convention
- One Dicey Issue
- Advanced Features of Functions
 - Function Declaration and Prototypes
 - Call by Value and Call by Reference
 - An Introduction to Pointers
 - Pointer Notation
 - Back to Function Calls
 - Conclusions
 - Recursion
- Adding Functions to the Library
- Summary
- Exercise

Knowingly or unknowingly we rely on so many persons for so many things. Man is an intelligent species, but still cannot perform all of life's tasks all alone. He has to rely on others. You may call a mechanic to fix up your bike, hire a gardener to mow your lawn, or rely on a store to supply you groceries every month. A computer program (except for the simplest one) finds itself in a similar situation. It cannot handle all the tasks by itself. Instead, it requests other program like entities—called 'functions' in C—to get its tasks done. In this chapter we will study these functions. We will look at a variety of features of these functions, starting with the simplest one and then working towards those that demonstrate the power of C functions.

What is a Function

A function is a self-contained block of statements that perform a coherent task of some kind. Every C program can be thought of as a collection of these functions. As we noted earlier, using a function is something like hiring a person to do a specific job for you. Sometimes the interaction with this person is very simple; sometimes it's complex.

Suppose you have a task that is always performed exactly in the same way—say a bimonthly servicing of your motorbike. When you want it to be done, you go to the service station and say, "It's time, do it now". You don't need to give instructions, because the mechanic knows his job. You don't need to be told when the job is done. You assume the bike would be serviced in the usual way, the mechanic does it and that's that.

Let us now look at a simple C function that operates in much the same way as the mechanic. Actually, we will be looking at two things—a function that calls or activates the function and the function itself.

```
main()  
{  
    message( ) ;  
    printf ( "\nCry, and you stop the monotony!" ) ;  
}  
message( )  
{  
    printf ( "\nSmile, and the world smiles with you..." ) ;  
}
```

And here's the output...

```
Smile, and the world smiles with you...  
Cry, and you stop the monotony!
```

Here, **main()** itself is a function and through it we are calling the function **message()**. What do we mean when we say that **main()** 'calls' the function **message()**? We mean that the control passes to the function **message()**. The activity of **main()** is temporarily suspended; it falls asleep while the **message()** function wakes up and goes to work. When the **message()** function runs out of statements to execute, the control returns to **main()**, which comes to life again and begins executing its code at the exact point where it left off. Thus, **main()** becomes the 'calling' function, whereas **message()** becomes the 'called' function.

If you have grasped the concept of 'calling' a function you are prepared for a call to more than one function. Consider the following example:

```
main()  
{  
    printf ( "\nI am in main" ) ;  
    italy( ) ;  
    brazil( ) ;  
    argentina( ) ;  
}
```

```
italy()  
{  
    printf ( "\nI am in italy" ) ;  
  
}  
brazil()  
{  
    printf ( "\nI am in brazil" ) ;  
}  
argentina()  
{  
    printf ( "\nI am in argentina" ) ;  
}
```

The output of the above program when executed would be as under:

```
I am in main  
I am in italy  
I am in brazil  
I am in argentina
```

From this program a number of conclusions can be drawn:

- Any C program contains at least one function.
- If a program contains only one function, it must be **main()**.
- If a C program contains more than one function, then one (and only one) of these functions must be **main()**, because program execution always begins with **main()**.
- There is no limit on the number of functions that might be present in a C program.
- Each function in a program is called in the sequence specified by the function calls in **main()**.

- After each function has done its thing, control returns to **main()**. When **main()** runs out of function calls, the program ends.

As we have noted earlier the program execution always begins with **main()**. Except for this fact all C functions enjoy a state of perfect equality. No precedence, no priorities, nobody is nobody's boss. One function can call another function it has already called but has in the meantime left temporarily in order to call a third function which will sometime later call the function that has called it, if you understand what I mean. No? Well, let's illustrate with an example.

```
main()  
{  
    printf ( "\nI am in main" );  
    italy();  
    printf ( "\nI am finally back in main" );  
}  
italy()  
{  
    printf ( "\nI am in italy" );  
    brazil();  
    printf ( "\nI am back in italy" );  
}  
brazil()  
{  
    printf ( "\nI am in brazil" );  
    argentina();  
}  
argentina()  
{  
    printf ( "\nI am in argentina" );  
}
```

And the output would look like...

```
I am in main
I am in italy
I am in brazil
I am in argentina
I am back in italy
I am finally back in main
```

Here, **main()** calls other functions, which in turn call still other functions. Trace carefully the way control passes from one function to another. Since the compiler always begins the program execution with **main()**, every function in a program must be called directly or indirectly by **main()**. In other words, the **main()** function drives other functions.

Let us now summarize what we have learnt so far.

- (a) C program is a collection of one or more functions.
- (b) A function gets called when the function name is followed by a semicolon. For example,

```
main()
{
    argentina();
}
```

- (c) A function is defined when function name is followed by a pair of braces in which one or more statements may be present. For example,

```
argentina()
{
    statement 1 ;
    statement 2 ;
    statement 3 ;
}
```

- (d) Any function can be called from any other function. Even **main()** can be called from other functions. For example,

```
main()
{
    message();
}
message()
{
    printf ( "\nCan't imagine life without C" );
    main();
}
```

- (e) A function can be called any number of times. For example,

```
main()
{
    message();
    message();
}
message()
{
    printf ( "\nJewel Thief!!" );
}
```

- (f) The order in which the functions are defined in a program and the order in which they get called need not necessarily be same. For example,

```
main()
{
    message1();
    message2();
}
message2()
{
    printf ( "\nBut the butter was bitter" );
}
```

```
}  
message1()  
{  
    printf ( "\nMary bought some butter" ) ;  
}
```

Here, even though **message1()** is getting called before **message2()**, still, **message1()** has been defined after **message2()**. However, it is advisable to define the functions in the same order in which they are called. This makes the program easier to understand.

- (g) A function can call itself. Such a process is called ‘recursion’. We would discuss this aspect of C functions later in this chapter.
- (h) A function can be called from other function, but a function cannot be defined in another function. Thus, the following program code would be wrong, since **argentina()** is being defined inside another function, **main()**.

```
main()  
{  
    printf ( "\nI am in main" ) ;  
    argentina()  
    {  
        printf ( "\nI am in argentina" ) ;  
    }  
}
```

- (i) There are basically two types of functions:

Library functions Ex. **printf()**, **scanf()** etc.

User-defined functions Ex. **argentina()**, **brazil()** etc.

As the name suggests, library functions are nothing but commonly required functions grouped together and stored in

what is called a Library. This library of functions is present on the disk and is written for us by people who write compilers for us. Almost always a compiler comes with a library of standard functions. The procedure of calling both types of functions is exactly same.

Why Use Functions

Why write separate functions at all? Why not squeeze the entire logic into one function, **main()**? Two reasons:

- (a) Writing functions avoids rewriting the same code over and over. Suppose you have a section of code in your program that calculates area of a triangle. If later in the program you want to calculate the area of a different triangle, you won't like it if you are required to write the same instructions all over again. Instead, you would prefer to jump to a 'section of code' that calculates area and then jump back to the place from where you left off. This section of code is nothing but a function.
- (b) Using functions it becomes easier to write programs and keep track of what they are doing. If the operation of a program can be divided into separate activities, and each activity placed in a different function, then each could be written and checked more or less independently. Separating the code into modular functions also makes the program easier to design and understand.

What is the moral of the story? Don't try to cram the entire logic in one function. It is a very bad style of programming. Instead, break a program into small units and write functions for each of these isolated subdivisions. Don't hesitate to write functions that are called only once. What is important is that these functions perform some logically isolated task.

Passing Values between Functions

The functions that we have used so far haven't been very flexible. We call them and they do what they are designed to do. Like our mechanic who always services the motorbike in exactly the same way, we haven't been able to influence the functions in the way they carry out their tasks. It would be nice to have a little more control over what functions do, in the same way it would be nice to be able to tell the mechanic, "Also change the engine oil, I am going for an outing". In short, now we want to communicate between the 'calling' and the 'called' functions.

The mechanism used to convey information to the function is the 'argument'. You have unknowingly used the arguments in the **printf()** and **scanf()** functions; the format string and the list of variables used inside the parentheses in these functions are arguments. The arguments are sometimes also called 'parameters'.

Consider the following program. In this program, in **main()** we receive the values of **a**, **b** and **c** through the keyboard and then output the sum of **a**, **b** and **c**. However, the calculation of sum is done in a different function called **calsum()**. If sum is to be calculated in **calsum()** and values of **a**, **b** and **c** are received in **main()**, then we must pass on these values to **calsum()**, and once **calsum()** calculates the sum we must return it from **calsum()** back to **main()**.

```
/* Sending and receiving values between functions */
main()
{
    int a, b, c, sum ;

    printf ( "\nEnter any three numbers " );
    scanf ( "%d %d %d", &a, &b, &c );

    sum = calsum ( a, b, c );
```

```
    printf ( "\nSum = %d", sum ) ;  
}  
  
calsum ( x, y, z )  
int x, y, z ;  
{  
    int d ;  
  
    d = x + y + z ;  
    return ( d ) ;  
}
```

And here is the output...

```
Enter any three numbers 10 20 30  
Sum = 60
```

There are a number of things to note about this program:

- (a) In this program, from the function **main()** the values of **a**, **b** and **c** are passed on to the function **calsum()**, by making a call to the function **calsum()** and mentioning **a**, **b** and **c** in the parentheses:

```
sum = calsum ( a, b, c ) ;
```

In the **calsum()** function these values get collected in three variables **x**, **y** and **z**:

```
calsum ( x, y, z )  
int x, y, z ;
```

- (b) The variables **a**, **b** and **c** are called ‘actual arguments’, whereas the variables **x**, **y** and **z** are called ‘formal arguments’. Any number of arguments can be passed to a function being called. However, the type, order and number of the actual and formal arguments must always be same.

Instead of using different variable names **x**, **y** and **z**, we could have used the same variable names **a**, **b** and **c**. But the compiler would still treat them as different variables since they are in different functions.

- (c) There are two methods of declaring the formal arguments. The one that we have used in our program is known as Kernighan and Ritchie (or just K & R) method.

```
calsum ( x, y, z )  
int x, y, z ;
```

Another method is,

```
calsum ( int x, int y, int z )
```

This method is called ANSI method and is more commonly used these days.

- (d) In the earlier programs the moment closing brace (**}**) of the called function was encountered the control returned to the calling function. No separate **return** statement was necessary to send back the control.

This approach is fine if the called function is not going to return any meaningful value to the calling function. In the above program, however, we want to return the sum of **x**, **y** and **z**. Therefore, it is necessary to use the **return** statement.

The **return** statement serves two purposes:

- (1) On executing the **return** statement it immediately transfers the control back to the calling program.
- (2) It returns the value present in the parentheses after **return**, to the calling program. In the above program the value of sum of three numbers is being returned.

- (e) There is no restriction on the number of **return** statements that may be present in a function. Also, the **return** statement need not always be present at the end of the called function. The following program illustrates these facts.

```
fun()  
{  
    char ch ;  
  
    printf ( "\nEnter any alphabet " );  
    scanf ( "%c", &ch );  
  
    if ( ch >= 65 && ch <= 90 )  
        return ( ch );  
    else  
        return ( ch + 32 );  
}
```

In this function different **return** statements will be executed depending on whether **ch** is capital or not.

- (f) Whenever the control returns from a function some value is definitely returned. If a meaningful value is returned then it should be accepted in the calling program by equating the called function to some variable. For example,

```
sum = calsum ( a, b, c );
```

- (g) All the following are valid **return** statements.

```
return ( a );  
return ( 23 );  
return ( 12.34 );  
return ;
```